barnes-vidit / **Jeevani**

Code Issues Pull requests Actions Projects Wiki Security

Commit 417b77d

barnes-vidit committed 3 weeks ago · ✓ 2 / 2

```
Update biographer logic and UI
```

biographer-update

1 parent 8c141d6 commit 417b77d

6 files changed +225 -13 lines changed

Top

Filter files...

- ai-service
  - main.py
  - rag_service.py
- client/src
  - components
    - TypewriterText.jsx
  - index.css
  - pages
    - Biographer.jsx
- server/routes
  - biographer.js

```
@@ -63,6 +63,25 @@ async def chat(request: ChatRequest):
63  63          except Exception as e:
64  64              raise HTTPException(status_code=500, detail=str(e))
65  65
    66  +  class GreetingRequest(BaseModel):
```

```
67  +     user_name: str
68  +     recent_uploads: list = []
69  +     last_chat: str = ""
70  +     on_this_day: list = []
71  +     current_date: str = ""
72  +
73  + @app.post("/chat/greeting")
74  + async def generate_greeting(request: GreetingRequest):
75  +     try:
76  +         print(f"Generating greeting for {request.user_name}")
77  +         context = request.dict()
78  +         greeting = rag.generate_greeting(context)
79  +         print(f"Greeting: {greeting}")
80  +         return {"greeting": greeting}
81  +     except Exception as e:
82  +         print(f"Greeting error: {e}")
83  +         raise HTTPException(status_code=500, detail=str(e))
84  +
66  85    @app.post("/ingest/file-process")
67  86    async def ingest_file_process(
68  87        background_tasks: BackgroundTasks,
```

```
@@ -15,7 +15,7 @@ def __init__(self):
15  15            try:
16  16                # Configure Gemini
17  17                genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
18      -             self.llm = genai.GenerativeModel('gemini-flash-latest')
    18  +             self.llm = genai.GenerativeModel('gemma-3-27b-it')
19  19
20  20                # Configure Pinecone
21  21                self.pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))

@@ -153,13 +153,64 @@ def delete_document(self, user_id: str, doc_id: str):
153 153               # Construct the filter. Note: We need to match the metadata structure used in ingest
154 154               # In ingest: base_id = f"{userId}_{docId}"
155 155               # Pinecone delete by filter is cleaner
    156 +             print(f"Attempting to delete vectors with filter: userId={user_id}, docId={doc_id}")
156 157               self.index.delete(
157 158                   filter={
```

```
158  159                          "userId": user_id,
159  160                          "docId": doc_id
160  161                      }
161  162                  )
     163  +               print(f"Deletion command sent to Pinecone for docId: {doc_id}")
     164  +               return True
     165  +           except Exception as e:
     166  +               print(f"Deletion command sent to Pinecone for docId: {doc_id}")
162  167                  return True
163  168              except Exception as e:
164  169                  print(f"Error deleting document vectors: {e}")
165  170                  return False
     171  +
     172  +     def generate_greeting(self, context: Dict) -> str:
     173  +         """
     174  +         Generate a contextual greeting based on user life-state.
     175  +         Context keys: user_name, recent_uploads (list), last_chat (str),
          on_this_day (list), current_date (str)
     176  +         """
     177  +         import random
     178  +
     179  +         user_name = context.get('user_name', 'Friend')
     180  +         uploads = context.get('recent_uploads', [])
     181  +         last_chat = context.get('last_chat', '')
     182  +         # Mood could be inferred from last chat or passed explicitly. For now, we
          let LLM infer from last_chat content.
     183  +         on_this_day = context.get('on_this_day', [])
     184  +         date_str = context.get('current_date', '')
     185  +
     186  +         # Construct Prompt with Tiered Logic
     187  +         prompt = f"""
     188  + You are Jeevani, a personal biographer. Your goal is to start a conversation with
          {user_name} ({date_str}).
     189  + Your tone is warm, empathetic, and curious—like an old friend catching up over
          coffee.
     190  +
     191  + **Context:**
     192  + - **Recent Uploads (Last 48h):** {uploads if uploads else "None"}
     193  + - **On This Day (Past Years):** {on_this_day if on_this_day else "None"}
     194  + - **Last Conversation Summary:** "{last_chat}"
     195  +
     196  + **Decision Logic (Prioritize in order):**
```

```
+ 1. **The Time Capsule:** If 'On This Day' has items, asking about that specific memory is PRIORITY #1. "I saw that X years ago today..."
+ 2. **The Detective:** If 'Recent Uploads' exist, ask a specific question about one of them. "I saw you added [File]..."
+ 3. **The Empath:** If 'Last Conversation' was sad, emotional, or unresolved, follow up on it gentle.
+ 4. **The Storyteller (Default):** If none of the above apply, pick ONE of these random angles to ask a deep life question:
+     - *Values*: A lesson they want to pass down.
+     - *Unsung Heroes*: A person who supported them silently.
+     - *Mischief*: A rule they broke in the past.
+     - *Pattern Matcher*: A habit you've noticed (make one up based on general life themes if no data).
+
+ **Constraint:**
+ - Generate ONLY the greeting/question.
+ - Keep it under 2 sentences.
+ - Be specific to the available context.
+ """
+         try:
+             response = self.llm.generate_content(prompt)
+             return response.text.strip()
+         except Exception as e:
+             print(f"Greeting generation failed: {e}")
+             return f"Hello {user_name}, I'm ready to document your story. What's on your mind today?"
```

```
@@ -6,11 +6,19 @@ export default function TypewriterText({ content, animate = false, onUpdate }) {
6  6      const hasAnimated = useRef(!animate); // If not animating, mark as done
7  7
8  8      useEffect(() => {
   9  +       // If already animated, show full content
9  10         if (hasAnimated.current) {
10 11             setDisplayedText(content);
11 12             return;
12 13         }
13 14
   15 +       // Only start if we are supposed to animate and haven't finished yet
   16 +       if (!animate) {
   17 +           setDisplayedText(content);
   18 +           hasAnimated.current = true;
```

```
+             return;
+         }
+
          let currentIndex = 0;
          const speed = 20; // ms per char

@@ -26,7 +34,11 @@ export default function TypewriterText({ content, animate = false, onUpdate }) {
          }, speed);

          return () => clearInterval(interval);
-     }, [content, animate, onUpdate]);
+         // Remove 'content' from dependency to avoid restart on content change (if minor)
+         // If content changes significantly, we might WANT to restart, but for this chat,
+         // the content is usually static once set.
+         // If streaming, this logic needs adjustment, but for now we are doing full-text receive.
+     }, [animate, onUpdate]); // Removed 'content' from dependency to prevent re-triggering

      return (
          <div className="prose prose-sm dark:prose-invert max-w-none">
```

client/src/components/TypewriterText.jsx

```
@@ -1,4 +1,3 @@
-
  @tailwind base;
  @tailwind components;
  @tailwind utilities;

@@ -64,7 +63,26 @@
    * {
      @apply border-border;
    }
+
    body {
      @apply bg-background text-foreground;
    }
  }
+
```

```
+ /* Custom Scrollbar */
+ ::-webkit-scrollbar {
+   width: 8px;
+   height: 8px;
+ }
+
+ ::-webkit-scrollbar-track {
+   @apply bg-transparent;
+ }
+
+ ::-webkit-scrollbar-thumb {
+   @apply bg-muted-foreground/30 rounded-full hover:bg-muted-foreground/50 transition-colors;
+ }
+
+ ::-webkit-scrollbar-corner {
+   @apply bg-transparent;
+ }
```

```
@@ -8,18 +8,55 @@ export default function Biographer() {
    const { getToken } = useAuth();
    const { user } = useUser();

+   // Use Vite env vars (Must be declared before use in useEffect)
+   const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000/api";
+
    // Chat State
-   const [messages, setMessages] = useState([
-       {
-           role: "assistant",
-           content: `Hello ${user?.firstName || "there"}! I'm ready to document your story. What's on your mind today?`,
-           animate: true
-       }
-   ]);
+   const [messages, setMessages] = useState([]); // Start empty, fetch greeting
    const [input, setInput] = useState("");
    const [loading, setLoading] = useState(false);
    const [isListening, setIsListening] = useState(false);

+   // Strict Lock: Use ref to prevent double-firing in Strict Mode
```

```
+    const hasFetchedGreeting = useRef(false);
+
+    // Fetch Greeting on Mount
+    useEffect(() => {
+        const fetchGreeting = async () => {
+            if (hasFetchedGreeting.current) return;
+            hasFetchedGreeting.current = true; // Lock immediately
+
+            setLoading(true); // Show loader during fetch
+            try {
+                const token = await getToken();
+                const res = await axios.get(`${API_URL}/biographer/greeting`, {
+                    headers: { Authorization: `Bearer ${token}` }
+                });
+
+                setMessages([{
+                    role: "assistant",
+                    content: res.data.greeting,
+                    animate: true
+                }]);
+            } catch (err) {
+                console.error("Failed to fetch greeting:", err);
+                // Fallback
+                setMessages([{
+                    role: "assistant",
+                    content: `Hello ${user?.firstName || "there"}! I'm ready to document your story. What's on your mind today?`,
+                    animate: true
+                }]);
+            } finally {
+                setLoading(false);
+            }
+        };
+
+        if (user && messages.length === 0) {
+            fetchGreeting();
+        }
+    }, [user, getToken, API_URL]); // Minimized dependencies
+
+
     // Scroll Refs
     const scrollRef = useRef(null);
     const autoScrollEnabled = useRef(true);
```

```
@@ -31,8 +68,7 @@ export default function Biographer() {
31  68      const [searchQuery, setSearchQuery] = useState("");
32  69      const [deletingId, setDeletingId] = useState(null);
33  70
34      -   // Use Vite env vars
35      -   const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5000/api";
    71  +
36  72
37  73      // Scroll Logic
38  74      const scrollToBottom = () => {
```

```
@@ -2,6 +2,7 @@ const express = require('express');
2   2   const router = express.Router();
3   3   const axios = require('axios');
4   4   const JournalEntry = require('../models/JournalEntry');
    5   + const Memory = require('../models/Memory'); // Import Memory model
5   6   const { requireAuth } = require('@clerk/express'); // Keep for now in case needed elsewhere, but unused here
6   7
7   8   // Manual Auth Middleware (fixes hanging requireAuth)
    @@ -26,7 +27,82 @@ const AI_SERVICE_URL = process.env.AI_SERVICE_URL || 'http://localhost:8000';
26  27
27  28  // DEBUG ROUTE
28  29  router.get('/version', (req, res) => {
29      -     res.json({ version: "1.2.0 - DEBUG MODE", message: "If you see this, the new code is loaded!" });
    30  +     res.json({ version: "1.3.0 - Greeting Engine Active", message: "If you see this, the new code is loaded!" });
    31  + });
    32  +
    33  + // @route   GET /api/biographer/greeting
    34  + // @desc    Get a context-aware greeting from AI
    35  + // @access  Private
    36  + router.get('/greeting', ensureAuthenticated, async (req, res) => {
    37  +     try {
    38  +         const { userId } = req.auth();
    39  +         const now = new Date();
    40  +
    41  +         console.log("Generating greeting context for:", userId);
    42  +
```

```
43  +         // 1. Recent Uploads (Last 48h)
44  +         const twoDaysAgo = new Date(now.getTime() - (48 * 60 * 60 * 1000));
45  +         const recentUploadsDocs = await Memory.find({
46  +             clerkUserId: userId,
47  +             createdAt: { $gte: twoDaysAgo }
48  +         }).limit(3).select('originalName fileType');
49  +
50  +         const recentUploads = recentUploadsDocs.map(d => d.originalName);
51  +
52  +         // 2. Last Chat Summary
53  +         const lastEntry = await JournalEntry.findOne({ userId }).sort({ date: -1
    });
54  +         let lastChat = "";
55  +         if (lastEntry && lastEntry.messages.length > 0) {
56  +             // Get last 2 messages
57  +             const lastMsgs = lastEntry.messages.slice(-2);
58  +             lastChat = lastMsgs.map(m => `${m.role}: ${m.content}`).join(" | ");
59  +         }
60  +
61  +         // 3. On This Day (Simple check for same Month/Day)
62  +         // Note: For MVP we might skip complex aggregation and just check if we
    have any 'JournalEntry' from previous years
63  +         // A robust "On This Day" usually requires MongoDB aggregation to match
    $month and $dayofMonth
64  +         const month = now.getMonth() + 1; // 1-12
65  +         const day = now.getDate(); // 1-31
66  +
67  +         const onThisDayPipeline = [
68  +             {
69  +                 $match: {
70  +                     clerkUserId: userId,
71  +                     $expr: {
72  +                         $and: [
73  +                             { $eq: [{ $month: "$createdAt" }, month] },
74  +                             { $eq: [{ $dayOfMonth: "$createdAt" }, day] },
75  +                             { $lt: [{ $year: "$createdAt" }, now.getFullYear()] }
    // Previous years only
76  +                         ]
77  +                     }
78  +                 }
79  +             },
80  +             { $limit: 2 },
81  +             { $project: { originalName: 1, year: { $year: "$createdAt" } } }
```

```
82  +         ];
83  +
84  +         const onThisDayDocs = await Memory.aggregate(onThisDayPipeline);
85  +         const onThisDay = onThisDayDocs.map(d => `Uploaded ${d.originalName} in
    ${d.year}`);
86  +
87  +         // 4. Call AI Service
88  +         const context = {
89  +             user_name: "Vidit", // We can get this from Clerk if we pass full user
    object, for now hardcode/placeholder
90  +             recent_uploads: recentUploads,
91  +             last_chat: lastChat,
92  +             on_this_day: onThisDay,
93  +             current_date: now.toDateString()
94  +         };
95  +
96  +         console.log("[Biographer] Sending Context to AI:", JSON.stringify(context,
    null, 2));
```

server/routes/biographer.js

```
    context);
99  +         res.json({ greeting: aiResponse.data.greeting });
100 +
101 +     } catch (error) {
102 +         console.error("Greeting Error:", error.message);
103 +         // Fallback
104 +         res.json({ greeting: "Hello Vidit! I'm ready to document your story. What's
    on your mind today?" });
105 +     }
30  106 });
31  107
32  108 // @route   GET /api/biographer/history
```

Comments 0

Lock conversation

Comment

Subscribe You're not receiving notifications from this thread.